

DOCUMENTACIÓN TRONVIER DE

ÍNDICE

1. Para qué sirve / se usa tkinter.....	1
2. Widgets básicos.....	2
Frame.....	2
Label.....	4
Button.....	6
MessageBox.....	8
Entry.....	11
Checkbutton.....	12
Radiobutton.....	14
Scale.....	16
Spinbox.....	18
Combobox.....	19
Progressbar.....	20
Text.....	22
Listbox.....	24
Scrollbar.....	25
Toplevel.....	26
3. Gestores de Geometría:.....	27
4. Orientación a objetos.....	29

1. Para qué sirve / se usa tkinter.

Tkinter es una herramienta de interfaces gráficas que viene incluida en Python y que te permite crear ventanas con elementos visuales como botones, campos de texto, menús e imágenes, con los que el usuario puede interactuar usando el ratón y el teclado.

Con Tkinter, ese mismo programa puede tener una ventana con botones, campos donde escribir, listas para elegir opciones y mensajes que aparecen al pulsar un botón, de forma que cualquier persona pueda usarlo sin saber nada de programación.

Para usarlo no hace falta instalar nada extra, basta con escribir `import tkinter` al inicio del código porque ya viene incluido con Python.

2. Widgets básicos.

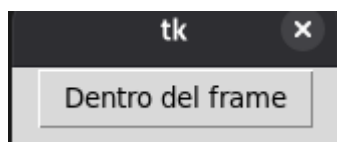
Frame

Frame es un widget que funciona como un contenedor que sirve para agrupar y organizar de mejor manera otros widgets. Frame es esencial para la creación de cualquier interfaz gráfica compleja en tkinter.

Aquí hay un ejemplo de código:

```
manual.py > ...
1  import tkinter as tk
2
3  app = tk.Tk()
4
5  frame = tk.Frame(app)      # contenedor hijo de la ventana principal
6  frame.pack()
7
8  tk.Button(frame, text="Dentro del frame").pack() # widget hijo del frame
9
10 app.mainloop()
```

Y así se vería:



Estos son **métodos** de frame:

```
frame.pack_propagate(False) # desactiva el ajuste automático de tamaño (con pack)
frame.grid_propagate(False) # lo mismo para grid
frame.winfo_width()         # ancho actual en píxeles
```

```

frame.winfo_height()      # alto actual en píxeles
frame.winfo_children()    # lista de widgets hijos
frame.configure(bg="red")  # cambiar opciones después de crearlo
frame.destroy()           # elimina el frame y todos sus hijos

```

Estos son **argumentos** de frame:

```

tk.Frame(
    app,
    bg="white",          # color de fondo del frame
    width=300,           # ancho en píxeles (necesita propagate=False para
respetarse)
    height=200,          # alto en píxeles (necesita propagate=False para
respetarse)
    bd=2,               # grosor del borde en píxeles
    relief="flat",       # borde:
"flat", "raised", "sunken", "groove", "ridge", "solid"
    cursor="arrow",      # cursor del ratón al pasar por encima
    padx=10,            # espacio interior horizontal
    pady=10,            # espacio interior vertical
    highlightthickness=1, # grosor del borde de enfoque
    highlightbackground="gray", # color del borde de enfoque sin foco
    highlightcolor="blue", # color del borde de enfoque con foco
)

```

Sin frame te verías obligado a usar el mismo gestor de geometría en toda la interfaz, pero frame permite utilizar dentro del contenedor un gestor diferente del que se utilice fuera de este. Por ejemplo puedes hacer esto:

```

manual.py > ...
1  import tkinter as tk
2
3  app = tk.Tk()
4
5  frame_top    = tk.Frame(app, bg="lightblue", height=60)
6  frame_center = tk.Frame(app, bg="white")
7  frame_bottom = tk.Frame(app, bg="lightgray", height=40)
8
9  frame_top.pack(fill="x")
10 frame_center.pack(fill="both", expand=True)
11 frame_bottom.pack(fill="x")
12
13 app.mainloop()
14

```

Y creas en la ventana principal tres frames en los que se puede guardar el resto de los widgets de una manera más organizada y fácil de controlar. También se puede utilizar este widget para crear varias ventanas que se puedan ocultar y mostrar otras.

```

manual.py > ...
1  import tkinter as tk
2
3  app = tk.Tk()
4  app.title("Cambio de páginas")
5  app.geometry("400x300")
6
7  def mostrar(frame):
8      frame.tkraise() # trae este frame al frente
9
10 # Frames (todos los frames)
11 frame1 = tk.Frame(app)
12 frame2 = tk.Frame(app)
13
14 for f in (frame1, frame2):
15     f.place(relwidth=1, relheight=1) # para que todos los frames ocupen el mismo espacio
16
17 # Contenido de cada página
18 tk.Label(frame1, text="Página 1").pack()
19 tk.Label(frame2, text="Página 2").pack()
20
21 # Para navegar entre frames
22 tk.Button(frame2, text="Página 1", command=lambda: mostrar(frame1)).pack()
23 tk.Button(frame1, text="Página 2", command=lambda: mostrar(frame2)).pack()
24
25 mostrar(frame1) # muestra la primera página al arrancar
26
27 app.mainloop()

```

Label

El widget Label es usado para mostrar texto o imágenes estáticas en una interfaz gráfica de usuario. La principal función de label es mostrar información al usuario como títulos, mensajes, o etiquetas descriptivas sin permitir que el usuario modifique dicha información directamente.

Label no tiene métodos propios, simplemente usa los métodos universales que ofrece tkinter.

La sintaxis básica de Label es la siguiente:

```
label = tk.Label(parent, opciones...)
```

- **label:** Es la variable donde guardamos el Label
- **parent:** Es el contenedor donde metemos los widgets
- **opciones:** Distintas opciones que le podemos meter a Label. Algunas de estas opciones son:
 - **text:** Es el texto que se va a mostrar en el Label.
Sintaxis: text = "texto"
 - **textvariable:** Obtiene el texto de una variable de Tkinter
Sintaxis variable: var = tk.StringVar(value="texto")
Sintaxis: textvariable=var
 - **image:** Especifica un objeto tk.PhotoImage (permite formato PNG, GIF, PPM y PGM).
Sintaxis variable: imagen = tk.PhotoImage(file="imagen.png")
Sintaxis: image=imagen

- **anchor:** La alineación. Puede ser: n, s, e, w, center...
Sintaxis: anchor="alineación"
- **font:** Fuente y tamaño del Label
Sintaxis: font=("fuente", tamaño_letra)
- **fg:** Color del texto del Label | **Sintaxis:** fg="color"
- **bg:** Color de fondo del Label | **Sintaxis:** bg="color"
- **padx:** Padding horizontal | **Sintaxis:** padx=padding
- **pady:** Padding vertical | **Sintaxis:** pady=padding
- **compound:** Cuando se usa **text** y **image**, especifica la posición de la imagen en relación a el texto. Si no se pone esta opción, se ignora el texto que se use. Las opciones que tiene compound son: left, right, top, bottom, center, none (por defecto). También permite las constantes de Tkinter, tk.LEFT, tk.RIGHT, etc. Ambas opciones hacen lo mismo.
Sintaxis: compound="posición"

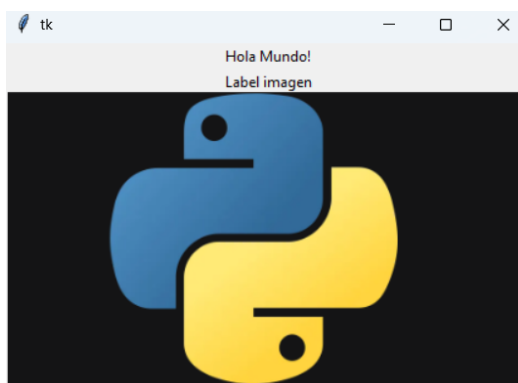
Ejemplo de uso de Label:

```

1  import tkinter as tk
2
3  # Contenedor para meter los widgets
4  app = tk.Tk()
5
6  # Sintaxis Label para mostrar texto
7  texto = tk.Label(app, text = "Hola Mundo!")
8  texto.pack()
9
10 # Variable con la imagen que se mostrará
11 img = tk.PhotoImage(file="python_logo.png")
12
13 # Sintaxis Label para mostrar imagen
14 imagen = tk.Label(app, text= "label imagen", image=img, compound= "bottom")
15 imagen.image = img
16 imagen.pack()
17
18 # Bucle principal de la aplicación
19 app.mainloop()

```

Resultado:



Button

Button es un widget que permite al usuario realizar una acción al clicar en él. Button suele ser usado para lanzar funciones o métodos.

La sintaxis de Button es: `boton=tk.Button(parent, text="texto", command=funcion)`

Aparte de las opciones generales como las de Label, Button tiene las siguientes opciones:

- **command:** La opción **command** sirve para llamar a una función o método cuando hacemos click en el botón. Para que el **command** no se active hasta que no se haga click en el botón se debe usar **lambda** antes.

Sintaxis: `command=funcion` | `command=lambda:funcion`

- **activebackground:** Esta opción sirve para que cuando hagamos click en el botón el color del botón cambie

Sintaxis: `activebackground="color"`

- **activeforeground:** Al igual que **activebackground**, se usa al hacer click en el botón, pero en vez de cambiar el fondo lo que cambia es el color del texto.

Sintaxis: `activeforeground="color"`

- **overrelief:** Esta opción se usa para que cuando pasemos el cursor del ratón por encima el botón tenga un borde. Tiene seis estados: **flat** (sin borde), **raised** (borde elevado 3D), **sunken**(hundido hacia adentro), **groove**(dos tonos de borde uno más oscuro y el lado contrario más claro), **ridge**(contrario a groove, el borde parece que sobresale), **solid**(borde sólido simple).

Sintaxis: `overrelief="estado"`

- **relief:** Es como **overrelief** pero este está siempre visible sin tener que pasar el cursor por encima. Comparte los mismos estados con **overrelief**.

Sintaxis: `relief="estado"`

- **state:** State se usa para controlar si el botón es interactivo o no, es decir si cuando pasamos el cursor o clicamos en el responde o no. Tiene tres estados:

- **normal:** Es el estado predeterminado del botón, responde a los clics y eventos del teclado mostrando los colores del **foreground** y del **background**.
- **active:** Se activa de manera automática cuando el cursor pasa sobre el botón usando **activeforeground** y **activebackground** para resaltar los colores.
- **disabled:** El botón se vuelve completamente insensible a eventos y clics. Para mostrar el estado generalmente se muestra en un color gris. Usa **disabledforeground** y **background** para mostrar esta nueva apariencia.

- **default:** Funciona parecido a **state**, pero **default** se usa para remarcar que ese botón es el principal o el que se debería usar por defecto. Tiene los mismos estados que **state**.

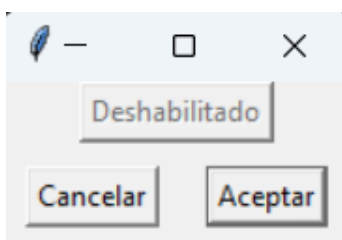
A parte tiene los siguientes métodos exclusivos, ya que los otros los hereda de la clase Widget y los comparten todos los widgets de Tkinter:

- **invoke():** Ejecuta el **command** del botón por el código igual que si el usuario hubiera hecho click en el botón.
- **flash():** Este método hace parpadear el botón alternando el color del **background** y del **activebackground**

Ejemplo código Button:

```
1 import tkinter as tk
2
3 app = tk.Tk()
4 app.title("Demostración state y default")
5
6 # Función que usaremos en los tres botones
7 def accion(nombre):
8     print(f"Pulsado: {nombre}")
9
10 # Botón deshabilitado
11 btn_dis = tk.Button(app, text="Deshabilitado", state="disabled", command=lambda: accion("deshabilitado"))
12 btn_dis.pack()
13
14 # Botón normal (sin destacar)
15 btn_cancel = tk.Button(app, text="Cancelar", default="normal", command=lambda: accion("cancelar"))
16 btn_cancel.pack(side="left", pady=10, padx=10)
17
18 # Botón por defecto (con borde extra)
19 btn_ok = tk.Button(app, text="Aceptar", default="active", command=lambda: accion("aceptar"))
20 btn_ok.pack(side="right", pady=10, padx=10)
21
22 app.mainloop()
```

Resultado código Button:



Messagebox

Messagebox es un **módulo de diálogos** de tkinter que se usa para **mostrar** al usuario **mensajes en ventanas emergentes**, ya sean simplemente **informativos**, de **advertencia**, **errores** o **preguntas**. Messagebox bloquea la ejecución de la aplicación hasta que el usuario no cierra la ventana.

Este módulo ofrece dos tipos de funciones, los **mensajes unidireccionales** y las **preguntas con respuesta**.

Los **mensajes unidireccionales** són los mensajes que únicamente **ofrecen información al usuario** sin necesitar una respuesta.

Los **mensajes unidireccionales** són:

- **showinfo()**: Muestra mensajes para poder informar al usuario. Muestra un icono de información.
- **showwarning()**: Muestra mensajes de advertencia al usuario. Muestra un icono de advertencia.
- **showerror()**: Muestra al usuario un mensaje de error. Muestra un icono de error.

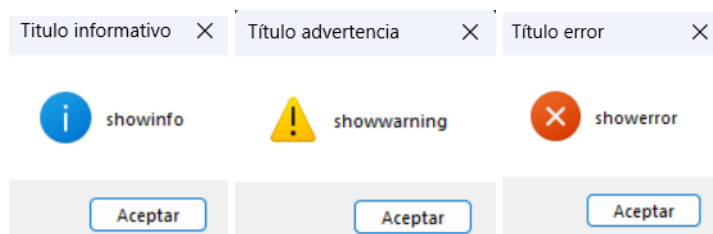
La sentencia para los tres es la misma, primero pone el título de la ventana, y después el mensaje que se muestra: `messagebox.metodo("titulo", "mensaje")`

Código mensajes unidireccionales:

```
from tkinter import messagebox

# Mensajes informativos
messagebox.showinfo("Titulo informativo", "showinfo")
messagebox.showwarning("Título advertencia", "showwarning")
messagebox.showerror("Título error", "showerror")
```

Resultado código:



Las **preguntas con respuesta**, són messagebox con **botones de valores booleanos** y **None** en algunos que según la elección del usuario **el programa actúa de una manera** o de **otra**. Van acompañadas de líneas de código para definir qué hacer cuando clickas un botón o otro.

Las preguntas con respuesta són:

- **askquestion("titulo", "string")**: Hace la pregunta y en vez de devolver un *booleano*, devuelve un *string*.
- **askyesno("titulo", "string")**: Es igual que **askquestion()** pero esta devuelve un *booleano*.
- **askyesnocancel("titulo", "string")**: Es igual que **askyesno()**, pero aparte de devolver un *booleano* también puede devolver *None*.
- **askokcancel("titulo", "string")**: Parecida a **askyesno()** pero esta pregunta si seguir o cancelar la ejecución.
- **askretrycancel("titulo", "string")**: Se suele usar en situaciones de error en la que se pregunta al usuario si quiere reintentar o no cuando se equivoca

Código preguntas con respuesta:

```
res = messagebox.askquestion("Askquestion", "¿Continuar?")
if res == "yes":
    print("Sí")
else:
    print("No")
```

```
res = messagebox.asksyesno("Askyesno", "¿Continuar?")
if res:
    print("Sí")
else:
    print("No")
```

```
res = messagebox.askyesnocancel("Askyesnocancel", "¿Guardar antes de salir?")
if res is True:
    print("Guardar y salir")
elif res is False:
    print("Salir sin guardar")
else:
    print("No salir")
```

```
res = messagebox.askokcancel("Askokcancel", "¿Ejecutar el proceso?")
if res:
    print("OK, ejecutando")
else:
    print("Cancelado")
```

```
res = messagebox.askretrycancel("Askretrycancel", "No se pudo conectar. ¿Reintentar?")
if res:
    print("Reintentando...")
else:
    print("Cancelado")
```

Resultado código preguntas con respuesta:

Askquestion ×

 ¿Continuar?

Askyesno ×

 ¿Continuar?

Askyesnocancel ×

 ¿Guardar antes de salir?

Askokcancel ×

 ¿Ejecutar el proceso?

Askretrycancel ×

 No se pudo conectar. ¿Reintentar?

Entry

Entry es un widget de Tkinter que permite al usuario poder ingresar o editar texto en una sola línea. Entry funciona como un campo de texto normal en el cual aparece el típico cursor intermitente para marcar donde se van a ingresar los caracteres. Entry permite recoger información, configurar su apariencia, ocultar datos y validar entradas.

Su sintaxis es: `entry = tk.Entry(parent, opciones...)`

Aparte de las opciones universales de Tkinter, Entry tiene algunas propias:

- **show:** Sirve para ocultar los datos que se escriben en ese momento, muy usado para ocultar contraseñas.
Sintaxis: `show="carácter"`
- **width:** Modifica el ancho del entry en caracteres. Por defecto el valor es 20.
Sintaxis: `width=numero(ej: 30)`

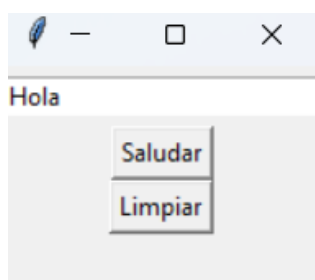
Los métodos más usados són:

- **.get():** Devuelve el texto que hay en el Entry en el momento que se ejecuta.
- **.insert(index, "texto"):** Inserta texto en la posición que se mande.
- **.delete(primer, último):** Elimina los caracteres del rango que se le indique.

Código Entry:

```
1 import tkinter as tk
2
3 def saludar():
4     nombre = entry.get()
5     label.config(text=f"Hola, {nombre}!")
6
7 def limpiar():
8     entry.delete(0, tk.END)
9     label.config(text="")
10
11 app = tk.Tk()
12
13 entry = tk.Entry(app, width=25)
14 entry.pack(pady=5)
15
16 tk.Button(app, text="Saludar",
17           command=saludar).pack()
18 tk.Button(app, text="Limpiar",
19           command=limpiar).pack()
20
21 label = tk.Label(app, text="")
22 label.pack(pady=5)
23
24 app.mainloop()
```

Resultado código:



Checkbox

Checkbox es un widget que de manera visual permite marcar una casilla, funciona como un botón normal con valores binarios, esto te permite activar o desactivar una opción de forma independiente.

Checkbox requieren de una variable de control para funcionar:

- **BooleanVar** -> guarda True/False.
`var_bool = tk.BooleanVar(value=True)`
- **IntVar** -> guarda 1/0 (o cualquier entero con onvalue/offvalue).
`var_int = tk.IntVar(value=0)`
- **StringVar** -> útil con onvalue/offvalue personalizados.
`var_str = tk.StringVar()`

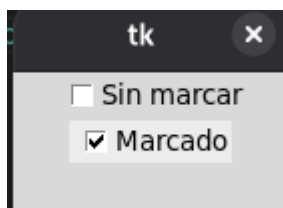
```
cb = tk.Checkbutton(root, variable=var_str, onvalue="sí",  
offvalue="no")
```

Dentro del checkbox deberá ir indicado cuál será la ruta padre o root donde estará guardado el checkbox, un texto que será lo que se escribirá al lado de la casilla y por último el nombre que se le haya asignado a la variable de control.

Esta es su estructura principal

```
manual.py > ...  
1 import tkinter as tk  
2  
3  
4 app = tk.Tk()  
5  
6 variable_control = tk.BooleanVar() # Variable de control  
7  
8 #cb es el nombre que se le ha asignado al objeto checkbox  
9 cb = tk.Checkbutton(app, text="Sin marcar", variable=variable_control)  
10 cb.pack()  
11  
12 variable_control2 = tk.BooleanVar(value=True) # Variable de control 2  
13  
14 cb2 = tk.Checkbutton(app, text="Marcado", variable=variable_control2)  
15 cb2.pack()  
16  
17 app.mainloop()
```

Y así se vería un checkbox luego de ejecutarse el código anterior:



Estos son **métodos** de `checkboxbutton`:

```
cb.select()      # Marca la casilla
cb.deselect()    # Desmarca la casilla
cb.toggle()      # Invierte el estado actual
cb.invoke()      # Simula un clic (ejecuta command y cambia estado)
cb.flash()       # Efecto visual de parpadeo
cb.configure(state="disabled") # Deshabilita
cb.configure(state="normal")   # Rehabilita
```

Estos son **argumentos** de `checkboxbutton`:

```
tk.Checkbutton(
    app,
    text="Acepto los términos", # texto visible
    variable=var,               # variable vinculada (BooleanVar, IntVar,
StringVar)
    onvalue=True,               # valor cuando está marcado
    offvalue=False,            # valor cuando no está marcado
    command=mi_funcion,         # función al cambiar estado
    selectcolor="blue",         # color del cuadro al marcar
    indicatoron=True,           # False → se convierte en botón toggle sin cuadro
    activebackground="lightgray", # color de fondo al hacer clic
    activeforeground="black",    # color del texto al hacer clic
    disabledforeground="gray",   # color del texto cuando state="disabled"
    wraplength=200,             # ancho máximo en px antes de salto de línea
)
```

Radiobutton

Radiobutton es un widget que te permite elegir entre varias opciones independientes, pueden ser dos, tres o el número de opciones que se desee.

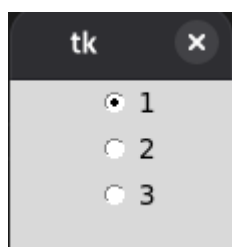
Al igual que checkbox se requiere de una variable para utilizarse, en cambio, aquí todos los radiobutton de un conjunto tendrán la misma variable.

Dentro del radiobutton deberá ir indicado cuál será la ruta padre o root donde estará guardado el radiobutton, un texto que será lo que se escribirá al lado de la casilla, el nombre que se le haya asignado a la variable de control y el valor que tendrá ese radiobutton.

Esta es su estructura principal:

```
manual.py > ...
1  import tkinter as tk
2
3
4  app = tk.Tk()
5
6  var = tk.StringVar(value="1") # este value marca por defecto el botón que
7                                #tenga el valor puesto.
8
9  boton1 = tk.Radiobutton(app, text="1", variable=var, value="1")
10 boton2 = tk.Radiobutton(app, text="2", variable=var, value="2")
11 boton3 = tk.Radiobutton(app, text="3", variable=var, value="3")
12
13 boton1.pack()
14 boton2.pack()
15 boton3.pack()
16
17 app.mainloop()
```

Y así se vería un radiobutton luego de ejecutarse el código anterior:



Estos son **métodos** de radiobutton:

```
rb.select()      # selecciona este botón (sin ejecutar command)
rb.deselect()    # deselecciona (raramente útil en radiobuttons)
rb.invoke()      # selecciona Y ejecuta command
rb.flash()       # efecto visual de parpadeo
rb.configure(state="disabled") # deshabilita
var.get()        # lee cuál opción está seleccionada
var.set("rust")  # cambia la selección por código
```

Argumentos de radiobutton:

```
tk.Radiobutton(
    app,
    text="Opción A",          # texto visible
    variable=var,             # variable COMPARTIDA por todo el grupo
    value="a",                # valor que toma la variable si se selecciona este
    command=mi_funcion,       # función al seleccionarlo
    selectcolor="blue",       # color del círculo al seleccionar
    indicatoron=True,         # False → botón de selección sin círculo
    activebackground="lightgray", # color de fondo al hacer clic
    activeforeground="black",  # color del texto al hacer clic
    disabledforeground="gray", # color del texto cuando state="disabled"
    wraplength=200,           # ancho máximo en px antes de salto de línea
)
```


Scale

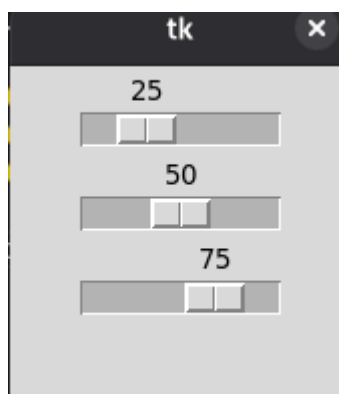
Scale es un widget que te permite mover un deslizador para marcar un valor que debe ser numérico dentro de un rango predefinido.

Dentro del scale deberá ir indicado cuál será la ruta padre o root donde estará guardado el scale, el nombre que se le haya asignado a la variable de control, el valor de inicio y final que tendrá ese scale, y por último la dirección del deslizador.

Esta es su estructura principal:

```
manual.py > ...
1  import tkinter as tk
2
3  app = tk.Tk()
4
5  var1 = tk.IntVar(value=25)
6  var2 = tk.IntVar(value=50)
7  var3 = tk.IntVar(value=75)
8
9  scale1 = tk.Scale(app, variable=var1, from_=0, to=100, orient="horizontal")
10 scale2 = tk.Scale(app, variable=var2, from_=0, to=100, orient="horizontal")
11 scale3 = tk.Scale(app, variable=var3, from_=0, to=100, orient="horizontal")
12
13 scale1.pack()
14 scale2.pack()
15 scale3.pack()
16
17 app.mainloop()
```

Y así se vería el scale luego de ejecutarse el código anterior:



Estos son **métodos** del scale:

```
scale.get()          # lee el valor actual
scale.set(60)        # cambia el valor por código
scale.configure(state="disabled") # deshabilita
scale.configure(state="normal")   # rehabilita
```

Argumentos de scale:

```
tk.Scale(
    app,
    variable=var,          # variable vinculada (IntVar o DoubleVar)
    from_=0,               # valor mínimo (from lleva _ porque from es
palabra reservada)
    to=100,                # valor máximo
    orient="horizontal",   # "horizontal" o "vertical"
    resolution=1,          # incremento mínimo entre valores
    tickinterval=25,       # cada cuánto mostrar marcas numéricas (0 = sin
marcas)
    length=300,            # longitud total del widget en píxeles
    showvalue=True,        # mostrar el valor actual encima del cursor
    label="Volumen",       # etiqueta de texto encima del scale
    command=mi_funcion,    # función al mover – recibe el valor como string:
def f(v): ...
    sliderlength=20,       # tamaño del cursor deslizante en píxeles
    width=15,              # grosor de la barra (no la longitud)
    digits=1,              # dígitos significativos mostrados
    troughcolor="lightgray", # color del canal por donde desliza el cursor
)
```

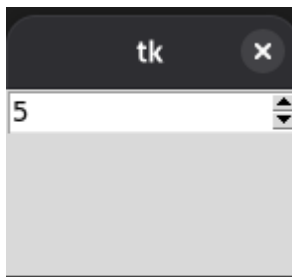
Spinbox

El Spinbox es un widget con un campo de entrada con flechas arriba/abajo que sirve para seleccionar un valor dentro de un rango o lista de opciones.

Esta es su estructura principal:

```
manual.py > ...
1  import tkinter as tk
2
3  app = tk.Tk()
4
5  var = tk.IntVar(value=5) # valor por defecto
6
7  spinbox = tk.Spinbox(app, from_=0, to=10, textvariable=var)
8  spinbox.pack()
9
10 app.mainloop()
```

Y así se vería el spinbox luego de ejecutarse el código anterior:



Estos son **métodos** de spinbox:

```
spinbox.get()           # lee el valor actual como string
spinbox.set(5)          # cambia el valor (solo en ttk.Spinbox)
spinbox.delete(0, "end") # borra el contenido
spinbox.insert(0, "valor") # inserta texto en la posición indicada
spinbox.configure(state="readonly") # solo lectura (flechas activas)
spinbox.configure(state="disabled") # deshabilitado total
```

Argumentos de spinbox:

```
tk.Spinbox(  
    app,  
    textvariable=var,          # variable vinculada – usa textvariable, NO  
    variable  
    from_=0,                   # valor mínimo del rango numérico  
    to=10,                     # valor máximo del rango numérico  
    increment=1,               # cuánto sube/baja el valor al pulsar las flechas  
    values=("a", "b", "c"),    # lista de opciones en lugar de rango (excluye  
    from_/to)                  # True → al llegar al máximo vuelve al mínimo  
    wrap=False,                # ancho del campo en caracteres  
    width=10,  
    state="normal",            # "normal", "disabled", "readonly" (flechas  
    activas, sin escritura)  
    command=mi_funcion,        # función al pulsar las flechas (no al escribir)  
    format="%.2f",              # formato numérico: "%.2f" muestra 2 decimales  
    justify="right",            # alineación del texto: "left", "right", "center"  
    buttonbackground="lightgray", # color de fondo de las flechas arriba/abajo  
    insertbackground="black",    # color del cursor de texto al escribir  
    readonlybackground="lightyellow", # color de fondo cuando state="readonly"
```

Combobox

Un Combobox es un widget que combina una entrada y una lista de opciones.

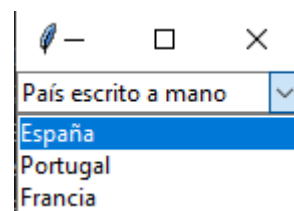
Sirve principalmente para dar al usuario la posibilidad de escribir su propia respuesta o de que elija de una selección. También se le puede bloquear la entrada para que sea obligatorio elegir una opción de la lista.

A diferencia del resto de widgets el combobox no se llama directamente del módulo tkinter se llama a través de ttk un submódulo de tkinter.

La sintaxis básica de Combobox es: `ttk.Combobox(contenedor, opciones...)`

Un ejemplo de Combobox:

```
1 import tkinter as tk
2 from tkinter import ttk
3 app = tk.Tk()
4
5 paises = ttk.Combobox(app)
6 paises['values'] = ('España', 'Portugal', 'Francia')
7 paises.pack()
8
9 app.mainloop()
```



Para usarlo le ponemos como argumento el elemento que lo contendrá, también le tendremos que dar unos valores esto se puede hacer de varias formas ya sea como se ve en el ejemplo o mediante el argumento “values”

Las opciones más comunes de Combobox son:

- **values**, te permite cambiar los valores que tendrá el Combobox, se ponen los distintos valores entre paréntesis y separados por comas.

`ttk.Combobox(contenedor, values=('Opción 1', 'Opción 2', 'Opción 3'))`

- **state**, define qué comportamiento tendrá la entrada del Combobox tendremos dos posibles opciones '**normal**' es el comportamiento predeterminado la entrada admitirá texto introducido por el usuario o '**readonly**' el usuario no podrá introducir nada en la entrada y tendrá que obligatoriamente coger una de las opciones, esta opción puede ser utilizada junto a un valor predeterminado del Combobox para que este nunca esté vacío.

`ttk.Combobox(contenedor, state='readonly')`

- **textvariable**, vincula el Combobox a una variable de Tkinter, la más utilizada es `tk.StringVar()`, principalmente sirve para que la variable se actualice automáticamente cuando el Combobox lo haga, después podemos recoger los datos directamente de la variable.

`ttk.Combobox(contenedor, textvariable=nombreVariable)`

- **height**, define el máximo número de filas que se verán al desplegar la lista, si se sobrepasa el límite se tendrá que ir desplazando de arriba hacia abajo para ver las distintas opciones se puede hacer con un “scrollbar” la opción por defecto es 10.

`ttk.Combobox(contenedor, height=15)`

Los **métodos** de Combobox son:

- **current(indice)**, devuelve el resultado que haya en el índice marcado con el argumento.

`nombreCombobox(0)` *#devuelve el primer elemento del combobox.*

- **set(cadena)**, cambia el texto del cuadro a la cadena que se le pase como argumento.

`nombreCombobox('Hola Mundo')` *#cambia el cuadro a Hola Mundo.*

- **get()**, devuelve el texto que el usuario tiene escrito o seleccionado en el momento que se ejecuta. No tiene argumentos.

`texto = nombreCombobox.get()`

También **hereda** los **métodos** del **tk.Entry**: **delete**, **insert** y **selection_clear**.

Progressbar

Un widget 'progressbar' nos permite dar feedback al usuario de que se está ejecutando una acción y podemos mostrarle una barra con el progreso. Tenemos dos tipos, las **determinadas** se usan cuando se sabe exactamente cuándo va a tardar en ejecutarse la tarea, la barra se irá llenando del 0% al 100% de forma controlada y las **indeterminadas** se usan cuando no se sabe cuánto va a tardar la acción y la barra simplemente se mueve de un lado a otro para indicar que está cargando. Está dentro del submódulo ttk.

La sintaxis básica es :

nombreProgressbar = ttk.Progressbar(contenedor, opciones...)

Un ejemplo de Progressbar:

```
1  import tkinter as tk
2  from tkinter import ttk
3
4  app = tk.Tk()
5
6  barra = ttk.Progressbar(app, mode='indeterminate')
7  barra.pack()
8  barra.start()
9
10 app.mainloop()
```

Quedaría de la siguiente forma:



Para usarlo necesitamos decirle un contenedor, de forma predeterminada es determinada, si queremos que sea indeterminada tendremos que ponerle la opción 'mode', por último tenemos que hacer que la barra empiece a avanzar con el método start.

Las opciones más comunes de Progressbar son:

- **mode**, indica si la barra será progresiva e irá avanzando de principio a fin '**determinate**' o será un cuadro rebotando de un lado a otro '**indeterminate**'.

```
ttk.Progressbar(contenedor, mode='indeterminate')
```

- **orient**, nos permite indicar si la barra será '**horizontal**' o '**vertical**'.

```
ttk.Progressbar(contenedor, orient='vertical')
```

- **length**, define el tamaño de la barra en píxeles.

```
ttk.Progressbar(contenedor, length=100)
```

- **maximum**, el total de unidades que representan el 100%, el por defecto es 100, es decir cada paso avanza un 1% si fuese 50 cada paso avanzaría 2%. A mayor valor más lento irá llenándose la barra y viceversa.

```
ttk.Progressbar(contenedor, maximum=20)
```

- **variable**, vincula la barra a una variable de Tkinter, por ejemplo a un tk.IntVar(), la barra se moverá en base al valor de esa variable.

```
ttk.Progressbar(contenedor, variable=nombreIntVar)
```

Los **métodos** más utilizados son:

- **step(amount = n)**, incrementa el valor de la barra en la cantidad (amount) que se le indique en n, puede ser entero o decimal, el por defecto es 1.0.

```
nombreProgressbar.step(5)
```

- **start(interval = 50)** el 'interval' se usa sobre todo en el modo indeterminado. La barra se moverá sola cada cierta cantidad de tiempo marcada en el parámetro 'interval' este número será la cantidad de milisegundos que tarda en moverse. Si no se pone parámetro simplemente hará que empiece la animación de la barra.

```
nombreProgressbar.start()
```

- **stop()**, detiene la animación que previamente se inició con start(). Reinicia el valor de la barra a 0

```
nombreProgressbar.stop()
```

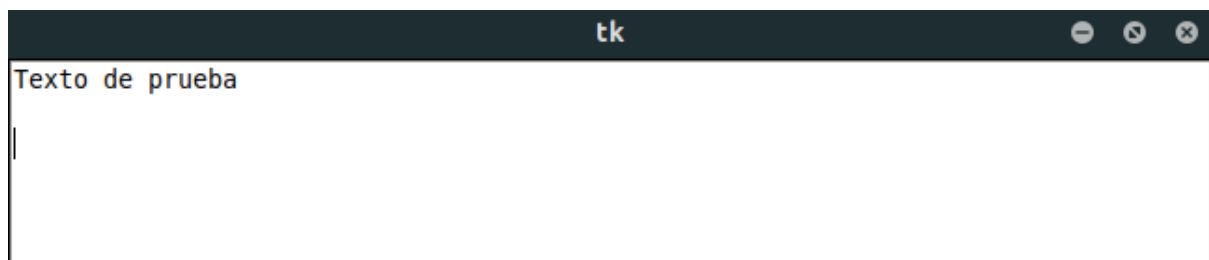

Text

El widget Text se utiliza para crear campos de entrada multilineas. Cuando se necesitan entradas más largas como comentarios o descripciones es mejor opción que un Entry. Este widget se llama directamente del módulo principal tk.

La sintaxis básica es:

nombreText = tk.Text(contenedor, opciones...)

```
1 import tkinter as tk
2
3 app = tk.Tk()
4
5 texto = tk.Text(app)
6 texto.insert('1.0', 'Texto de prueba')
7 texto.pack()
8
9 app.mainloop()
```



Los índices para usar este widget se hacen con dos números separados por un punto, por ejemplo 1.1, el primer número significa la línea y el segundo la columna, se puede usar los valores especiales de tkinter como tk.END

Las opciones más comunes del Text:

- width, es el ancho del cuadro indicado en cantidad de caracteres, por defecto es 80.

tk.Text(contenedor, width=40)

- height, es el alto del cuadro en cantidad de líneas, por defecto es 24.

tk.Text(contenedor, height=10)

- wrap, controla qué pasa cuando la línea de texto llega al final, hay dos opciones, 'word' que corta la palabra completa a la siguiente línea, 'char' que corta directamente el la letra y rompe la palabra y por último 'none' que simplemente hace que el texto siga avanzando a la derecha.

tk.Text(app, wrap='word')

Los métodos más utilizados para text son:

- insert(indice, texto), inserta el texto en el índice que se le indique (formato del índice: fila.columna)

```
nombreText.insert("1.0", "Texto inicial")
```

- get(inicio, fin), en este widget es obligatorio poner desde donde a donde queremos leer. Para obtener todo podemos usar 1.0 y tk.END.

```
todo_el_texto = nombreText.get("1.0", tk.END)
```

- delete(inicio, fin), borra el texto que se encuentre entre las dos posiciones indicadas para borrar todo el Text podemos hacer igual que en el get usando 1.0 y tk.END.

```
nombreText.delete("1.0", tk.END)
```

Listbox

Funcionalidad:

El Listbox es un widget que nos sirve principalmente como una lista de opciones seleccionables en la que podemos seleccionar uno o varios elementos, insertar o borrar dinámicamente diferentes opciones dentro de dicha lista. Siendo su principal uso para listar datos de manera visual.

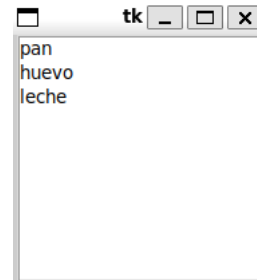
Recibe como parámetro el padre del que vaya a formar parte.

```
import tkinter as tk

app = tk.Tk()

lista_de_la_compra = tk.Listbox(app)
lista_de_la_compra.pack()
lista_de_la_compra.insert(tk.END, 'pan', 'huevo', 'leche')

app.mainloop()
```



Métodos:

.activate(index): Recibe el índice de un elemento dentro de la lista y sirve para poner el foco visual en dicho elemento sin necesidad de seleccionarlo.

.curselection(): Guarda el índice del elemento que está seleccionado dentro de la lista en el instante en que se ejecuta el método

.delete(first, last=None): Borra dentro de la lista los elementos que van desde el primer índice hasta el segundo especificado, si solo se especifica uno se borrará únicamente el elemento que pertenezca dicho índice especificado.

.get(first, last=None): Sirve para obtener el texto de uno o varios elementos que estén en la posición de los índices insertados, devolviéndolos en una lista de str.

.index(index): Sirve para convertir palabras clave especiales de Tkinter o coordenadas de clics en el número de índice que le corresponde en ese momento.

.insert(index, * elements): Sirve para agregar dentro de la lista los diferentes elementos nombrados, siendo el primer parámetro el índice donde se insertará dicho elemento o su posición y el resto los elementos a insertar.

.itemcget(index, option): sirve para consultar la configuración visual de un elemento específico de la lista.

.itemconfigure(index, cnf=None, **kw): sirve para cambiar o modificar el diseño visual de un elemento específico de la lista usando su número de índice.

.see(index): Sirve para hacer scroll a través de la lista hasta el índice indicado.

Scrollbar

En Tkinter, una Scrollbar funciona mediante un sistema de comunicación bidireccional (de ida y vuelta) con el widget que quiere controlar como puede ser una Listbox.

Puede recibir como parámetros:

orient: Define la dirección de la barra (`tk.VERTICAL` o `tk.HORIZONTAL`).

command: Conecta la barra al widget para mover su contenido (ej: `lista.yview`).

width: Controla el grosor de la barra en píxeles.

```
import tkinter as tk

app = tk.Tk()

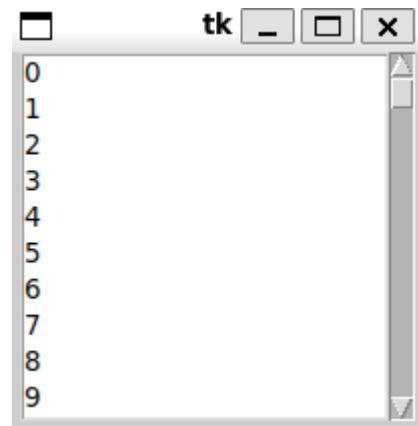
barra = tk.Scrollbar(app)
barra.pack(side=tk.RIGHT, fill=tk.Y)

lista = tk.Listbox(app, yscrollcommand=barra.set)
lista.pack(side=tk.LEFT)

barra.config(command=lista.yview)

for i in range(100):
    lista.insert(tk.END, i)

app.mainloop()
```



Métodos:

.set(first, last): Actualiza el tamaño y la posición del indicador (lo usa el widget sobre el que actúa el scrollbar automáticamente).

.get(): Devuelve la posición actual del indicador como una tupla de dos fracciones entre 0.0 y 1.0.

Toplevel

El Toplevel es un widget que sirve para crear una segunda ventana flotante que es independiente . Recibe como parámetro el padre al que va a pertenecer la nueva ventana, normalmente va a pertenecer a la ventana principal.

```
import tkinter as tk

app = tk.Tk()
app.title('Ventana Principal')

ventana_nueva = tk.Toplevel(app)
ventana_nueva.title("Ventana Secundaria")

app.mainloop()
```



TopLevel al ser básicamente otra ventana más como puede ser la principal, posee los mismos parámetros y los mismos métodos que cualquier otra ventana.

Métodos:

- .title(nuevo_titulo):** Establece el título de la ventana.
- .geometry(ancho * alto):** Establece las dimensiones de la ventana.
- .destroy():** cierra la ventana definitivamente.
- .grab_set():** Bloquea la ventana de atrás hasta que se cierre esta.

3. Gestores de Geometría:

En Tkinter, un gestor de geometría sirve para organizar, posicionar y colocar los widgets dentro de la ventana o frame indicado a la hora de crear el widget.

Con ellos podemos organizar y estructurar todos los diferentes tipos de widgets que queramos utilizar , dándole orden y estilo a la ventana.

Tenemos 3 tipos diferentes de gestores de Geometría:

- **Pack():** Es el más básico y sencillo de utilizar. Básicamente consiste en ir apilando widgets en bloques de forma secuencial uno debajo del otro.

Recibe como parámetros:

- **side:** Posición que ocupa el widget cuando se apila.
tk.TOP (por defecto, arriba),
tk.BOTTOM (abajo),
tk.LEFT (izquierda),
tk.RIGHT (derecha).
- **fill:** Cuanto ocupara dentro del bloque el widget.
tk.X (se estira horizontalmente),
tk.Y (se estira verticalmente),
tk.BOTH (se estira en ambas direcciones)
tk.NONE (por defecto, no se estira).
- **expand:** Recibe True o False. Si se pone en True, el widget le pide a la ventana que le asigne todo el espacio libre que sobre.
- **padx / pady :** Establece una separación en el eje x o y , entre diferentes widgets en pixeles.
- **Grid():** Es el gestor de geometría más potente, limpio y utilizado de tkinter. Esto se debe a que a la hora de utilizarlo puedes decidir cómo organizar los diferentes widgets, según su fila y columna, las cuales comienzan contando desde el 0.
Recibe como parámetros:

- **row:** Número de la fila en la que se colocará el widget.
- **column:** Número de columna en la que se colocará el widget.
- **sticky:** Sirve para alinear el widget dentro de su propia casilla si esta es más grande que él.
N (Norte/Arriba),
S (Sur/Abajo),
E (Este/Derecha),
W (Oeste/Izquierda).
- **rowspan:** Indica cuántas filas de alto va a ocupar el widget.
- **columnspan:** Indica cuántas columnas de ancho va a ocupar el widget.
- **padx / pady:** Establece una separación en el eje x o y , entre diferentes widgets en pixeles.

- **Place():** Es el gestor de geometría que te permite tener un mayor control sobre la pantalla. Organizando los diferentes widgets por píxeles tomando las coordenadas 0,0 como origen.

Tiene 2 categorías de parámetros:

- **Posicionamiento absoluto:**

Fijan el widget en una posición concreta

x: Distancia en píxeles desde el borde izquierdo de la ventana hacia la derecha.

y: Distancia en píxeles desde el borde superior de la ventana hacia abajo.

width: Define el ancho exacto que tendrá el widget en píxeles.

height: Define el alto exacto que tendrá el widget en píxeles.

- **Posicionamiento relativo:**

Establecen una posición en función del tamaño de la ventana (con valores del 0 al 1)

relx: Posición horizontal relativa al ancho de la ventana.

rely: Posición vertical relativa al alto de la ventana

relwidth y relheight: Hacen que el tamaño del widget cambie proporcionalmente al tamaño de la ventana.

anchor: Define qué parte del widget se engancha en la coordenada indicada

tk.N (North / Norte): Centrado arriba.

tk.S (South / Sur): Centrado abajo.

tk.E (East / Este): Alineado a la derecha.

tk.W (West / Oeste): Alineado a la izquierda

tk.CENTER (Centro): Centrado perfecto tanto vertical como horizontalmente.

4. Orientación a objetos

Cuando una aplicación de Tkinter crece, escribir todo el código de forma secuencial (lineal) se vuelve difícil de mantener y organizar. La mejor práctica es estructurar el programa utilizando **Programación Orientada a Objetos (POO)**.

Las cosas más importantes a tener en cuenta son:

- **Heredar de `tk.Tk` (Subclase):** Al hacer que tu clase sea una subclase de `tk.Tk`, tu clase se *convierte* en la ventana principal. Ya no necesitas escribir `app = tk.Tk()`, porque la propia instancia de tu clase ya es la ventana.
- **El uso de `super().__init__()`:** Es obligatorio llamarlo en la primera línea del constructor. Esto inicializa la ventana de Tkinter que estás heredando y activa todo su funcionamiento interno.
- **El contenedor ahora es `self`:** Al crear los widgets dentro de la clase, el primer argumento ya no es una variable externa (como `app` o `ventana`), sino `self`, que significa "esta misma ventana".
- **Cuándo usar `self` en los widgets:** * Si necesitas leer o modificar un widget desde otra función de la clase (por ejemplo, leer el texto de un `Entry`), **debes** guardarlo con `self.nombre_widget`.
 - Si el widget solo sirve para mostrarse y no vas a interactuar con él más adelante (como un botón o una etiqueta estática), puedes crearlo como una variable normal sin `self..`

```
1 import tkinter as tk
2 from tkinter import ttk
3
4 class Aplicacion(tk.Tk):
5     def __init__(self):
6         super().__init__()
7
8         self.entrada = ttk.Entry(self)
9         self.entrada.pack()
10
11        boton = ttk.Button(self, text="Obtener texto", command=self.mostrar)
12        boton.pack()
13
14        def mostrar(self):
15            print(self.entrada.get())
16
17 app = Aplicacion()
18 app.mainloop()
```